UE23CS320A- Project Phase - 1

End Semester Assessment

Project Title : AI-Assisted Test Case Generation for Mobile and Web UI/UX Applications
Project ID : 101
Project Guide : Dr.Farida Begam

Project Team : Navya G N PES2UG23CS372
Nidhi K PES2UG23CS383
Nikita Kolathaya PES2UG23CS387
Sandeep PES2UG23CS525

Introduction

Modern web and mobile applications are becoming increasingly dynamic, interactive, and frequently updated.
Because of this fast evolution, **Quality Assurance teams struggle to design and maintain functional test cases** for every new interface change. Traditional automation tools such as Selenium and Playwright help execute tests, but they still depend on humans to create and update the scripts.

With the growth of AI - especially **Large Language Models (LLMs)** - it has now become possible to analyse UI screens, understand user flows, and generate structured test cases automatically. This project explores how AI can assist in **end-to-end functional test case generation** for real applications by combining browser automation, LLM reasoning, and vision-based UI understanding.

Motivation

We selected this problem because:

- **LLMs have recently shown strong reasoning abilities** to interpret UI screens and generate structured actions, but there is no practical system that applies this for real web applications.
- QA teams must repeatedly analyze screens and workflows because **existing automation tools cannot infer UI behavior on their own**.
- Modern UIs change rapidly, and script-based automation **breaks even for minor UI updates**..
- There is a clear academic and industry need for a system that can **autonomously explore an application, understand how it works, and generate functional test cases** similar to a human tester.
- Building such a system allows us to contribute to an active research direction while delivering a practical tool that QA teams can actually use.

Context

Problem Context

- Functional test cases are still written manually in most organizations.
- Script-based automation breaks frequently whenever the UI changes.
- Current tools can execute tests, but **do not understand the UI** well enough to generate test cases on their own.
- Existing workflows are fragmented across multiple tools for exploration, documentation, and execution.

Source:https://idealink.tech/blog/understanding-software-testing-costs-development-breakdown

Problem Statement

Today's applications evolve rapidly with dynamic interfaces, frequent updates, and diverse user workflows. However, the process of creating functional test cases is still largely manual. QA teams must explore each page, analyze the interface, determine valid user actions, and write test cases step-by-step - a process that is **time-consuming**, **repetitive**, and **difficult to scale**.

While automation frameworks like Selenium and Playwright have improved test execution, they still **rely on human-written script**s. Every interaction - clicking buttons, entering data, validating state transitions - must be explicitly defined and coded. This leads to **high maintenance overhead**, especially when dealing with fast-changing UIs where even minor UI changes can break test scripts.

Problem Statement

The current testing workflow is also fragmented. Testers use one tool to inspect the DOM, another to capture UI behaviors, a third to write test cases, and yet another to execute them. This disjointed process introduces **friction**, **increases chances of error**, and **slows down the feedback loop** between development and quality assurance.

What's needed is a more intelligent, unified, and autonomous approach that can generate and execute meaningful test cases by understanding how the application behaves - just like a human tester, but much faster.

Problem Statement

Proposed Solution:

We’ aim to build a system that automatically explores a live website, generates test cases, and runs them, with no human effort needed

Exploration: Using Playwright, the system opens the site, extracts the DOM, detects clickable/input elements, and records each step in a memory log.

Test Generation: A Large Language Model (LLM) reads the memory log and outputs structured test cases in JSON — each with clear steps and expected results.

Execution: The test cases are converted to Playwright scripts and run automatically. Results are logged with pass/fail status and errors.

Comparison System: We’re also building a second, vision-based system using YOLO + OCR to generate tests from screenshots. This helps us compare text-based vs visual reasoning performance.

Abstract and Scope

Main Objective : To design and develop an AI-assisted testing system that:

- Connects to a live web application,
- Explores and understands the interface, and
- Generates and executes a sequence of actions to explore the UI.
- Automatically generates functional test cases for each page visited.

In Scope

- Functional test case generation
- Browser automation using Playwright
- LLM integration for analyzing web pages and generating tests.
- Exploration loop that records each visited page and user path.
- Simple dashboard interface for input and results display.

APPROACH

Architecture A: LLM-based (DOM-driven)

- **Browser Controller (Playwright):** Opens pages, extracts DOM, optional screenshots.
- **LLM Module:** Reads DOM JSON and suggests next actions + generates test cases.
- **Main Orchestrator:** Runs exploration loop and compiles memory log into test cases.
- **Dashboard:** Takes URL input, shows generated test cases and results.

Architecture B: Vision-based (YOLO + OCR)

- **Browser Controller:** Captures screenshots for CV analysis.
- **YOLO/OCR Module:** Detects UI components visually and extracts labels.
- **LLM Module:** Converts detected elements into structured test cases.
- **Main Orchestrator + Dashboard:** Same as Architecture A.

Tech Stack

Backend: Python or Node.js

Automation: Playwright

LLM Reasoning: GPT-4 Turbo / GPT-3.5 / Claude 3.5

Optional Vision: YOLO + OCR for UI detection (second architecture for comparison)

Frontend Dashboard: HTML/CSS/JS or React

Data Storage: JSON logs or SQLite

Environment: WSL / Linux

Suggestions from Review - 1

- **Initial project scope was unclear and appeared overly broad.**

  The review panel felt the system attempted to solve too many problems at once (UX, accessibility, functional testing, RL agents, etc.).

- **Lack of clarity on input/architecture source.**

  Reviewers asked how the system would get data, does the RL agents implementation.

- **Unclear distinction between in-scope vs out-of-scope tasks.**

  The original proposal was to make fully autonomous testing agent.

- **Required deeper literature survey.**

  Panel recommended to in scope to make it “AI - Assisted Test Case Generation.”

Actions Taken After Review 1

- **Scope refined to only focus on functional test-case generation using LLM + Playwright.**

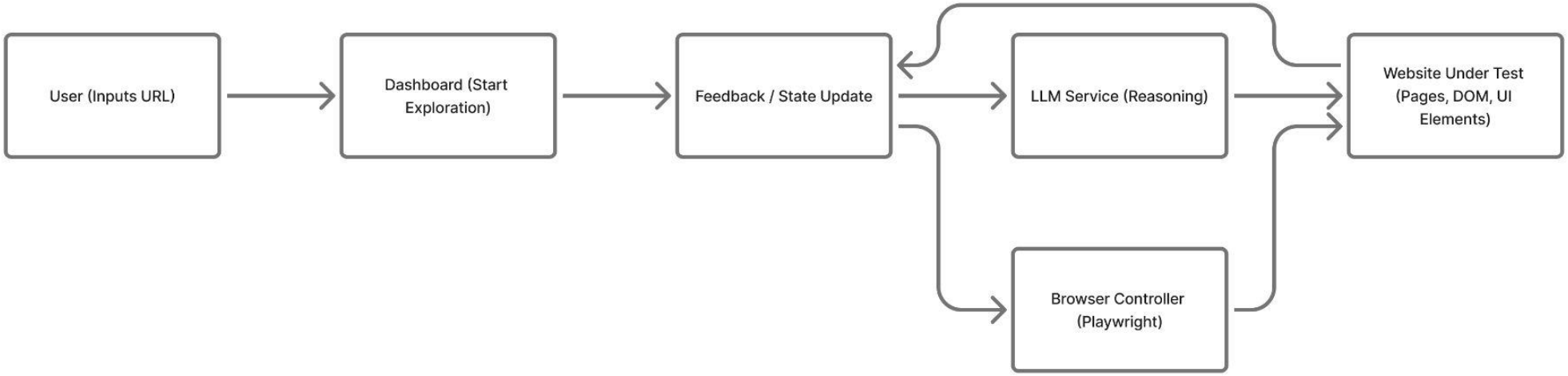
  Removed UX analysis, accessibility checks, Reinforcement learning.

- **system clarified.**

  System now uses open-source LLMs; DOM + screenshots collected via Playwright during exploration.

- **Clear architecture defined.**

  Browser Controller → LLM Service → Exploration Loop → Test Generator → Dashboard.



Challenges and Research Gap

1. Limited Use of LLMs for UI Understanding & Test Case Generation

LLMs haven't been fully utilized to understand UI structure (DOM + screenshot) and generate functional tests.

2. Absence of LLM-Driven Automated Exploration

No framework currently integrates Playwright with LLM reasoning to decide page navigation actions.

3. Incomplete UI Interpretation (DOM-only or Vision-only limitations)

DOM-only methods miss visual context; vision-only methods miss structural metadata.

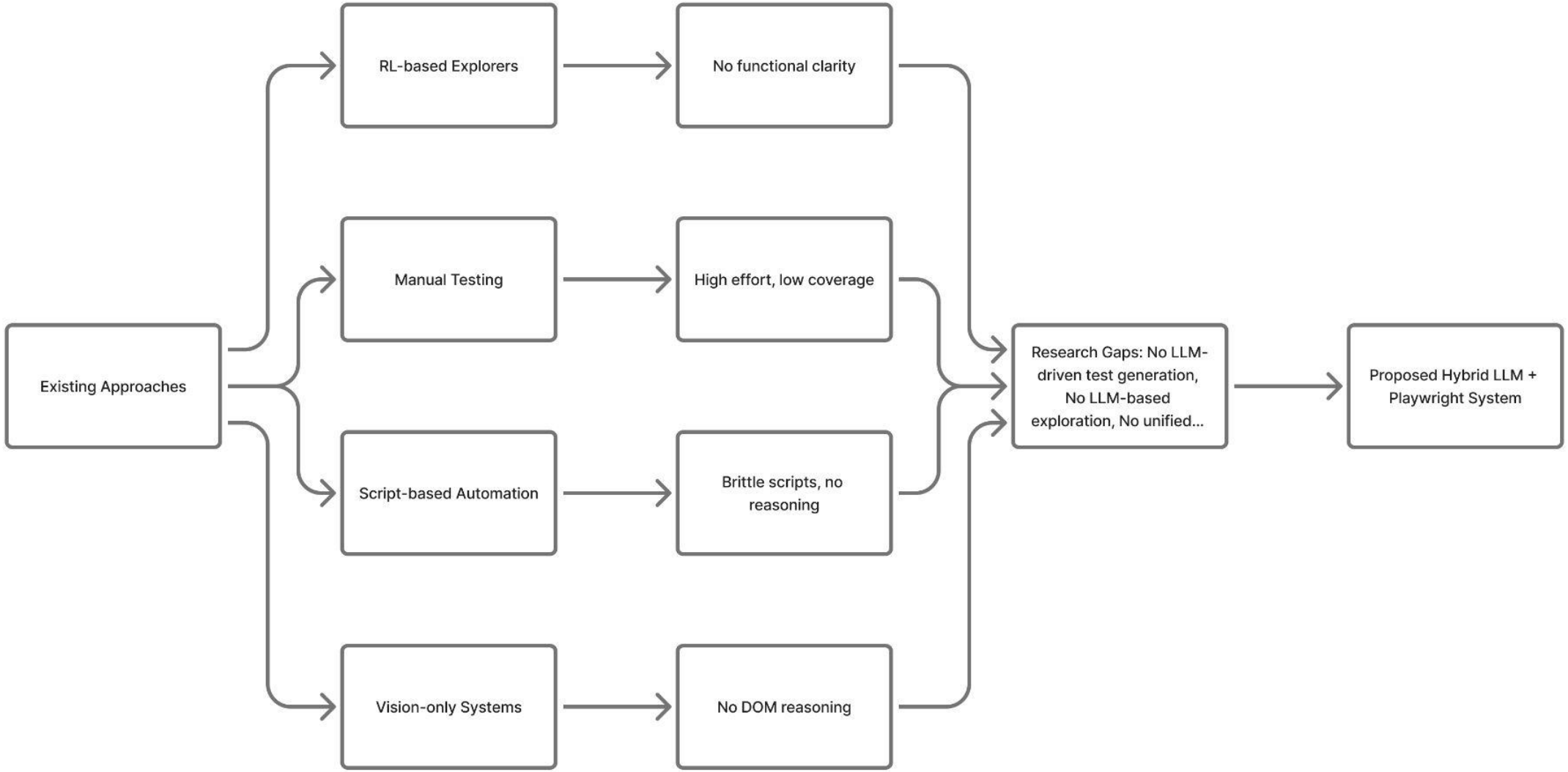
4. Scalability & Generalization Issues

Models trained on limited datasets fail when UIs change, or new websites are tested.

5. Missing End-to-End Architecture

No complete flow from capturing UI → analyzing → generating tests → executing them automatically.

Challenges and Research Gap



Objectives

1. To develop an Automated UI Exploration module that primarily uses Playwright based exploration engine to navigate through web and mobile interfaces to discover pages, actions, workflows etc, by capturing the state (DOM and screenshots).A memory log of visited pages and interface states is maintained to avoid repetition.
2. To implement an LLM-powered test generator (Architecture A) with the captured UI state and memory logs as input to generate functional test cases by giving prompts to the LLM.

Objectives

3. To implement a vision template based analyzer (Architecture B) that processes screenshots to detect UI elements and uses templates to produce test cases.

4. To implement automated conversion of structured test specifications into Playwright test scripts.

5. To integrate and execute generated tests and compare architectures A and B by measuring metrics such as exploration depth, functional coverage, accuracy etc.

Title :	J. Yoon, R. Feldt, and S. Yoo, "Intent-Driven Mobile GUI Testing with Autonomous Large Language Model Agents (DroidAgent)," in *Proc. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023.
Methodology	Proposed a **multi-agent cognitive architecture (Planner, Actor, Observer, Reflector)** using GPT-3.5 and GPT-4 to perform **intent-driven GUI testing** on Android apps. The system perceives the GUI hierarchy (XML tree), plans user-like "intents" (goals), executes actions through ADB, observes results, and reflects to improve future decisions. Introduces **memory modules (short-, long-, and spatial-term)** and an **intent hierarchy** for goal-oriented testing instead of random exploration.
Findings/Research	Demonstrated that LLM-based agents can autonomously generate **realistic user tasks** and execute them on mobile apps without prior training. Achieved higher **activity and feature coverage** than baseline random or script-based explorers.
Future work	It could be extended to handle **multi-app workflows**, i.e., tasks that **span across multiple apps**.
Datasets used	Tested on a **set of real Android applications** (20–30 apps) from open-source repositories like **F-Droid**
Gaps	Relies heavily on **textual GUI metadata**, so it may miss visual clues. Lacks **true reinforcement learning or self-improvement**; reflection memory is heuristic, not continuously optimized.

Paper title	**Vision-driven Automated Mobile GUI Testing via Multimodal Large Language Model** Z. Liu, C. Li, C. Chen, J. Wang, B. Wu, Y. Wang, J. Hu, and Q. Wang, in *Proc. IEEE/ACM Intl. Conf. on Software Testing, Verification and Validation (ICST)*, July 2024.
Methodology Used	**Multimodal Prompting:** Combines GUI screenshots with view-hierarchy metadata to form “vision prompts.”(UIAutomator for view-hierarchy extraction (XML)) **Function-Aware Explorer:** LLM-guided exploration that chooses next action(click/scroll/input) based on app functions, using history of previous screens and actions. **Logic-Aware Bug Detector:** Segments the interaction session into meaningful logical steps and asks the MLLM to detect anomalies using graphs and ADB **Implementation:** Uses GPT-4 multimodal abilities + Android UIAutomator to extract hierarchy structures and execute actions.
Result	Achieves ~57–61% activity coverage and 55–59% code coverage, exceeding baselines. Detects 83 real non-crash bugs across 105 apps. Reports precision 50–76% and recall 42–64% in bug detection.
Limitations	Vision-only reasoning **struggles** when screens have icon-only widgets or missing labels. **Dataset size is limited** - benchmarks with ~100 screens may not represent complex production-scale apps. **MLLM issues:** High compute cost, hallucinations, inconsistent reasoning. **Dependency on accurate view-hierarchy extraction:** Missing or noisy metadata reduces performance. **Workflow depth limitation:** Long, multi-screen flows remain difficult; segmentation and recovery are still imperfect.

Title	**Are Autonomous Web Agents Good Testers?**, A. Chevrot, A. Vernotte, J.-R. Falleri, X. Blanc, and B. Legeard in *Proc. of the IEEE/ACM Intl. Conf. on Automated Software Engineering (ASE)*, July 2025.
Methodology	Constructed a controlled benchmark using **3 offline web applications** and a suite of **113 manual test cases** (spanning both passing and failing scenarios). Implemented and compared two distinct Autonomous Test Agents (ATAs): 1. **SeeAct-ATA:**SeeAct-ATA is a very simple Autonomous Test Agent created by modifying the SeeAct Web Agent so it follows test-case steps and performs assertions using a single LLM prompt. It relies almost entirely on the LLM for reasoning, grounding, and action selection, making it basic but easy to implement. 2. **PinATA:** is a more advanced ATA that separates responsibilities into three components—an orchestrator, an actor, and an assertor—to improve planning, grounding, and verification. It uses state-of-the-art techniques like Set-of-Marks and Agent-as-a-Judge, making it far more accurate and reliable than SeeAct-ATA.
Key Results	**Performance Gap:** The baseline agent performed poorly (24% success), while the advanced **PinATA achieved ~60% correct verdicts** with high specificity (94%). **Conclusion:** Agents are promising but currently lack the robustness to fully replace human testers or scripted automation for complex tasks.
Limitations	**Strategic Exploration:** Agents struggle when the "happy path" is blocked. Unlike humans, they often fail to adapt their strategy when they encounter unexpected pop-ups or errors, leading to test failures.

Paper Title	**Deep Reinforcement Learning for Automated Web GUI Testing** Zhiyu Gu, Chenxu Liu, Guoquan Wu, Yifei Zhang, ChenXi Yang, Zheheng Liang, Wei Chen, and Jun Wei, in *Proc. IEEE/ACM Intl. Conf. on Automated Software Engineering (ASE)*, 2025.
Methodology	**Web_RLED Framework:** Uses a **Grid-based Action Value** network to approximate Q-values across the UI surface (splitting the screen into grids). **Action Discriminator:** A novel online module that learns to distinguish interactive elements (buttons, inputs) from static text during the crawl, without predefined rules.
Results	**Scale & Precision:** Tested on **50 Top Real-World Apps** (not just demo sites). Discovered **695 unique failures** (crashes, broken layouts). **Performance:** Achieved significantly higher **Code Coverage** and **State Coverage** than traditional crawlers (like Crawljax) and standard RL tools (like WebExplor).
Limitations	**Resource Intensity:** The Deep RL model requires significantly more **computational power and time** to train per website compared to lightweight random testing. **Cold Start:** The agent initially struggles to find deep states until the neural network has gathered enough interaction data to form a valid policy.

Title	**Using Vision LLMs for UI Testing,** L. Tan, D. Dai, and J. Lyu, in *Proc. ACM Conference on Intelligent User Interfaces (IUI)*, Jan 2025.
Methodology	**Visual-First Testing:** Uses Multimodal LLMs (like GPT-4o or Gemini Pro Vision) to "look" at the UI screenshot rather than reading the HTML code. **Grid Overlay (GridGPT):** To solve the AI's inability to click precise pixels, the system overlays a numbered grid on the screenshot, allowing the LLM to say "Click cell 42" instead of "Click division button-submit".
Key Findings	**Zero-Code Automation:** The approach successfully decoupled tests from the underlying code. The VLM could identify buttons like "Settings Icon" purely by visual context, even if the developer changed the entire underlying HTML structure.
Limitations	**Latency & Cost:** Processing high-resolution images via LLMs is significantly slower and more expensive than running a lightweight Selenium script. **Spatial Hallucination:** Without the grid overlay, Vision LLMs still frequently hallucinate precise coordinates for small UI elements.

Title :	5] R. Feldt, S. Kang, J. Yoon, and S. Yoo,“Towards Autonomous Testing Agents via Conversational Large Language Models,”in Proc. 38th IEEE/ACM Int. Conf. Automated Software Engineering (ASE),Luxembourg, pp. 1688–1691 ,2023.
Methodology	The paper proposes a taxonomy that categorizes how LLMs can be used in testing—from simple code completion to fully autonomous conversational testing agents. It illustrates this through a conversational workflow where GPT-4 helps generate and refine unit tests, showing how dialogue enables iterative improvement.
Findings/Research	LLMs can successfully generate meaningful tests, refine them through conversation, and even challenge developer assumptions, improving test quality.
Future work	Building a full autonomous agent (SOCRATEST) that can plan tests, use external tools, access memory, and execute tests without human intervention.
Datasets used	No structured dataset was used; instead, the evaluation is based on real conversational interactions between developers and GPT-4.
Gaps	Current LLMs lack true autonomy; they need middleware to interact with tools, run tests, or manage memory beyond a single conversation. LLMs also struggle with planning, consistency, tool-use reliability, and may generate incorrect assumptions without verification.

Paper title	"AgentA/B — Automated and Scalable Web A/B Testing with Interactive LLM Agents", Z. Liu, C. Li, C. Chen, J. Wang, B. Wu, Y. Wang, J. Hu, and Q. Wang, in *Proc. IEEE/ACM Intl. Conf. on Software Testing, Verification and Validation (ICST)*, July 2024.
Methodology Used	**Goal:** Use LLM-powered autonomous agents to simulate users and run A/B tests on *real* websites to reduce dependence on live human traffic. **System pipeline:** four modules — (1) LLM agent persona generation, (2) testing preparation (traffic/assignment), (3) iterative agent–web interaction (environment parser → LLM decision → action execution), and (4) post-test analysis (aggregated metrics, exportable logs). Implementation uses ChromeDriver + Selenium for browser control, JSON-based environment parsing to feed LLMs, and an action execution module with retry/fallback logic for failures. **Agent design:** generate large numbers of personas (authors generated 100k personas, sampled 1k agents for experiments), each agent has an intention/goal and acts in a loop: perceive (JSON snapshot), plan (LLM), act (browser), repeat. Sessions capped by action/time thresholds; action space includes Search, Click_product, Click_filter_option, Purchase, Stop. **Case study setup:** E-commerce/Amazon experiment comparing control (full left filter panel) vs treatment (reduced filter options by similarity threshold). Used Claude 3.5 Sonnet backend, distributed execution (16 nodes), each session capped at 20 actions.
Result	**Behavioral alignment:** Agents are more *goal-directed* (shorter sessions, fewer exploratory actions) but show similar conversion-like signals. **A/B sensitivity:** AgentA/B detected condition-dependent behavior (treatment → more clicks/purchases), aligning in direction with human A/B results. Authors reported agents could reproduce known A/B effects and surface differences across interface variants.
Limitations	**Human variability not fully captured:** agents simulate goal-directed behavior but lack affective/uncertainty signals (fatigue, hesitation) and some meta-cognitive behaviors. That can bias simulations toward more efficient paths than humans.

Paper Title	***Challenges of End-to-End Testing with Selenium WebDriver and How to Face Them: A Survey*** M. Leotta, B. García, F. Ricca, and J. Whitehead, in *Proc. 2023 IEEE Conf. Software Testing, Verification and Validation (ICST)*,2023.
Methodology	**Systematic Literature & Practitioner Review:** Surveyed **academic papers**,hundreds of **industrial resources,** Extracted **common root causes** of test failures . Focused on identifying **specific root causes** of test decay, Proposed **solutions** validated by practitioners, Compared practices such as Waits, Page Object Model, locator strategies, etc.
Results	**Root Cause Analysis:** Identified **AJAX/Asynchronous loading** as the primary cause of "Flakiness" and **DOM volatility** (dynamic IDs) as the cause of "Brittleness." **Verified Solutions:** Confirmed that **Page Object Pattern** reduces maintenance by ~30% and **Explicit Waits** are the only reliable fix for sync issues.
Limitations	**High Skill Barrier:** The recommended solutions are **architectural** (e.g., building abstraction layers), which requires senior-level coding skills, making them difficult for manual testers to adopt. **No "Silver Bullet":** The study concludes that no single tool completely eliminates flakiness; it requires constant human maintenance.

Dataset Exploration

Our system does not use any pre-existing dataset. It builds its own dataset in real-time through autonomous exploration of web apps using **Playwright**. This dynamic dataset reflects the live structure and behavior of the UI. During exploration, three major components are collected:

- **Page Data**
  - The DOM is extracted from each visited page.
  -All visible interactive elements are captured:
  tags (button, input, a), selectors, text content, attributes (id, class, name).

Dataset Exploration

- **Memory Log (Exploration Log)**

  - This is the core dataset of the system.
  - For every action taken by the system, the log stores:

    `step`: action number
    `from_url`: current page
    `action`: type (click, fill, navigate)
    `selector`: element targeted
    `to_url`: destination page
    `timestamp`: exact time

  It captures full navigation flow, making it useful for reconstructing workflows and verifying UI logic.

Dataset Exploration

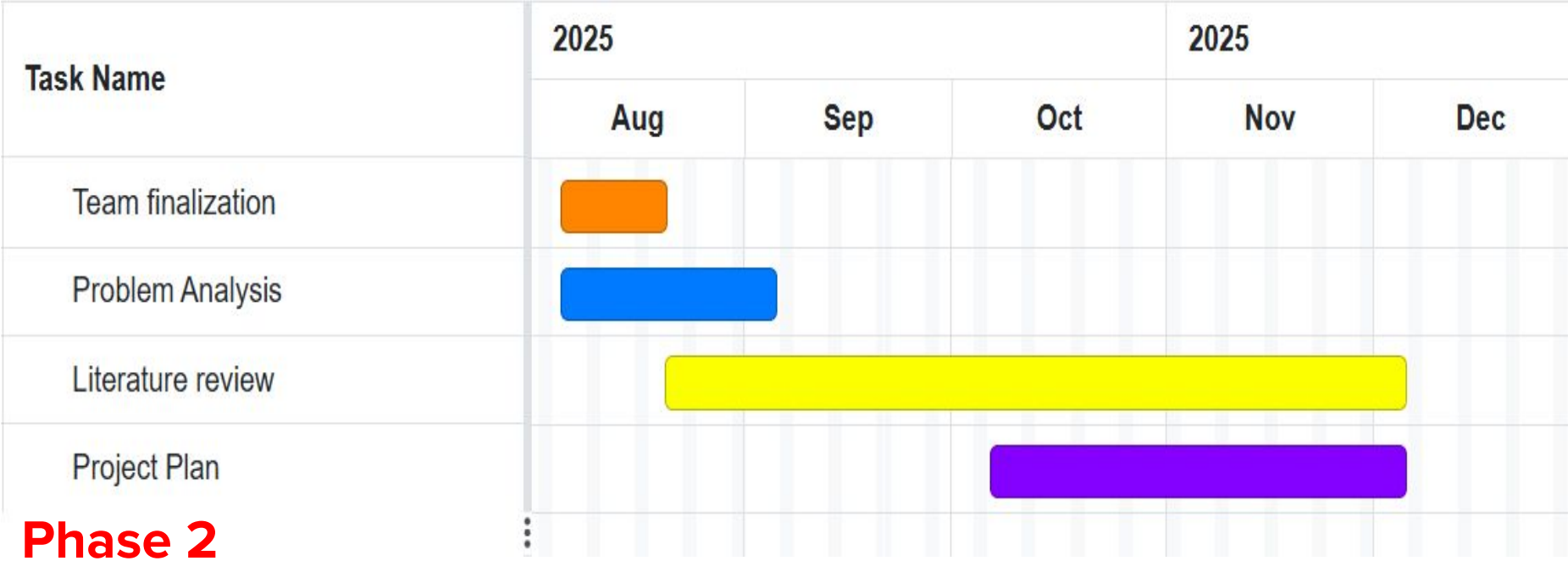
Generated Test Cases

- The Memory Log is passed to a **Large Language Model (LLM)**.

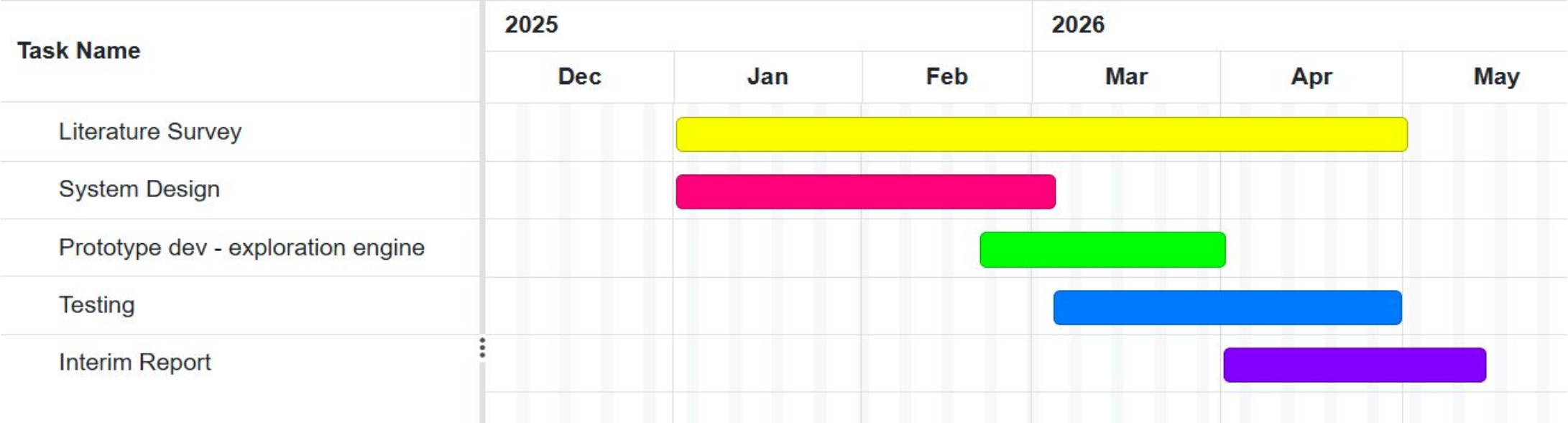
- LLM outputs structured test cases in JSON format. Each test includes:
  - id: unique test identifier
  - objective: purpose of the test
  - steps[]: ordered sequence of UI actions (click/fill) with selectors and values
  - expected_result: final expected behavior or destination page

Application: https://demoqa.com/
https://parabank.parasoft.com/parabank/api-docs/index.html

Phase 1

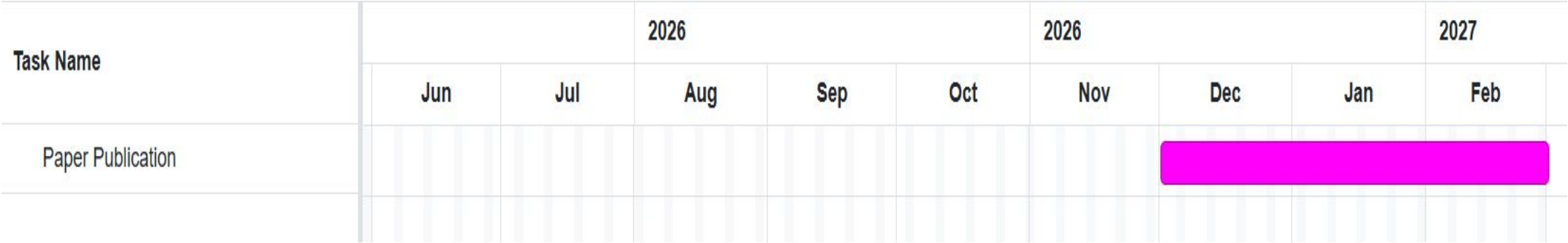


Phase 2



Phase 3

Phase 4



Expected Deliverables

- **Capstone I: Literature survey and architecture**
- **Capstone II: Developing the 2 architectures.**
  - Complete exploration loop
  - LLM-based test-case generator that converts exploration logs to structured executable Playwright test scripts.
  - Initial evaluation on multiple websites with metrics like number of tests generated, success rate, and coverage.

Capstone III: Evaluation, Optimization & Paper Publication

Improved prompts and rules for test generation and assertion validation.

Quantitative evaluation: precision/recall of generated test cases vs manually written baseline.

Final dashboard visualization of results and full research paper (methodology, experiments, results, discussion, future work).

Capstone IV: Research Paper & Publication

- A comprehensive **research paper** is prepared, detailing the system's architecture, methodology, and the benchmark results from Capstone III.
- The final paper is submitted to a **Scopus-indexed conference or journal**.

References

[1] J. Yoon, R. Feldt, and S. Yoo, “Intent-Driven Mobile GUI Testing with Autonomous Large Language Model Agents (DroidAgent),” in *Proc. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023.

[2] Z. Liu, C. Li, C. Chen, J. Wang, B. Wu, Y. Wang, J. Hu, and Q. Wang, “Vision-Driven Automated Mobile GUI Testing via Multimodal Large Language Models (VisionDroid),” in *Proc. IEEE/ACM Intl. Conf. on Software Testing, Verification and Validation (ICST)*, July 2024.

[3]A. Chevrot, A. Vernotte, J.-R. Falleri, X. Blanc, and B. Legeard ,“Are Autonomous Web Agents Good Testers?,” in *Proc. of the IEEE/ACM Intl. Conf. on Automated Software Engineering (ASE)*, July 2025.

[4] L. Tan, D. Dai, and J. Lyu, “Using Vision LLMs for UI Testing,” in *Proc. ACM Conference on Intelligent User Interfaces (IUI)*, Jan 2025.

[5] R. Feldt, S. Kang, J. Yoon, and S. Yoo,“Towards Autonomous Testing Agents via Conversational Large Language Models,”in *Proc. 38th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*,Luxembourg, pp. 1688–1691 ,2023.

[6] Dakuo Wang, Ting-Yao Hsu, Yuxuan Lu, Limeng Cui, Yaochen Xie, William Headean, Bingsheng Yao, Akash Veeragouni, Jiapeng Liu, Sreyashi Nag, and Jessie Wang, “AgentA/B — Automated and Scalable Web A/B Testing with Interactive LLM Agents,” *arXiv preprint* arXiv:2504.09723, 2024.

References

[7] M. Leotta, B. García, F. Ricca, and J. Whitehead, “Challenges of End-to-End Testing with Selenium WebDriver and How to Face Them: A Survey,”in *Proc. 2023 IEEE Conf. Software Testing, Verification and Validation (ICST)*,2023.

[8] Zhiyu Gu, Chenxu Liu, Guoquan Wu, Yifei Zhang, ChenXi Yang, Zheheng Liang, Wei Chen, and Jun Wei, “Deep Reinforcement Learning for Automated Web GUI Testing,” in *Proc. IEEE/ACM Intl. Conf. on Automated Software Engineering (ASE)*, 2025.

Thank
You